

Curso Inicial Front-End

Clase 2: Git y GitHub: El flujo de trabajo profesional

Módulo 1: Arquitectura y Maquetación Moderna (HTML y CSS)



Presentación

Programar es un deporte de equipo. En esta clase aprenderás **Git**, la herramienta que permite a miles de desarrolladores trabajar en el mismo proyecto sin destruirse el trabajo entre sí. Entenderás cómo viajar en el tiempo a versiones anteriores de tu código y cómo conectar tu PC con la nube de **GitHub**.



Objetivos

- Configurar tu identidad en **Git** y dominar el flujo local (Add, Commit).
- Comprender la diferencia técnica entre **Git** y **GitHub**.
- Crear, conectar y subir tu primer repositorio profesional a internet.



Bloques Temáticos

- Tema 1: Conceptos de control de versiones y sistema distribuido.
 - ¿Qué es el Control de Versiones (Git)?
 - El ADN de un Cambio: El COMMIT y el SHA-1
- Tema 2: Inicialización de repositorios, áreas de trabajo y commits.
 - Puesta en marcha del entorno local y el flujo de trabajo profesional
 - Conectando con la Nube (GitHub)
 - Buenas Prácticas para Commits

Tema 1: Conceptos de control de versiones y sistema distribuido.

1. ¿Qué es el Control de Versiones (Git)?

Imagina que estás escribiendo la tesis de tu universidad o un código complejo. Sin una herramienta como **Git**, seguramente terminarías guardando tus archivos con nombres como `tesis.doc`, `tesis_final.doc`, `tesis_final_ESTA_SI.doc`.

Hacer esto en el ámbito de la programación es un caos absoluto propenso al desastre. Para resolver este problema, la industria del software adoptó de forma universal sistemas que registran los cambios de manera inteligente, permitiendo experimentar con seguridad y resguardar el valor del trabajo.

- **Git** es un **Sistema de Control de Versiones Distribuido (DVCS)**.

A diferencia de los sistemas antiguos donde un servidor central guardaba el historial, en **Git** tú tienes una copia completa de toda la historia en tu computadora. Puedes viajar en el tiempo o crear versiones alternativas de tu proyecto incluso en un avión sin internet.

- **El Repositorio (Repo):** Cuando le decimos a **Git** que "vigile" una carpeta, esa carpeta se convierte en un *Repositorio*. Internamente, **Git** crea una carpeta oculta llamada `.git` (El "Cerebro"). Allí se guarda todo el historial de cambios en forma de base de datos comprimida.

WARNING

¡Nunca borres la carpeta oculta `.git`! Si la borras, la carpeta vuelve a ser una carpeta normal de Windows/Mac y perderás todo tu historial de versiones para siempre.

 **Recurso Oficial:** [Documentación Oficial de Git](#) | [Descargar Git](#) | [GitHub Docs](#)

Comandos: Configuración Pro: **Git** Bash por Defecto En Windows, la consola estándar (CMD) no es la mejor para **Git**. Sigue estos pasos para usar **Git** Bash dentro de VS Code:

- Abre la terminal en VS Code (**Ctrl + ñ**).
- En la esquina superior derecha de la terminal, haz clic en la flechita hacia abajo (junto al signo **+**).
- Elige **Select Default Profile** (Seleccionar perfil predeterminado).
- Selecciona **Git Bash**. Ahora, cada vez que abras una terminal, tendrás el poder de Linux en tu Windows.

3 Formas de abrir la terminal rápidamente:

- **Atajo:** Presiona **Ctrl + ñ** (o **Ctrl + `** backtick).
- **Menú:** Pestaña superior **Terminal** -> **New Terminal**.
- **Mouse:** Pon el mouse en el borde inferior de VS Code y arrastra hacia arriba cuando el cursor cambie de forma.

El Ecosistema del Código

Antes de pasar a la práctica en la terminal, es fundamental comprender la naturaleza del control de versiones y conocer las herramientas que dominan la industria actual.

Aunque **GitHub** es el rey, existen otras plataformas profesionales que podrías encontrar en empresas:

- **GitHub:** La red social del código (la que usaremos).
- **GitLab:** Muy usada en empresas privadas por sus herramientas de integración.
- **Bitbucket:** La preferida de equipos que usan Jira y herramientas de Atlassian.

Para balancear el uso de estas tecnologías y entender su alcance en el día a día, analicemos sus pros y contras:

✓ Ventajas (Pros)	✗ Desventajas (Contras)
Viaje en el tiempo: Puedes restaurar versiones viejas si rompes tu código.	Curva de Aprendizaje: Los comandos de terminal pueden ser intimidantes al principio.
Trabajo en equipo: Permite que 10 personas editen el mismo archivo sin pisarse.	Gestión de Conflictos: Resolver problemas cuando dos personas editan la misma línea es tedioso.
Offline: Todo el historial está en tu disco, no necesitas internet para usar Git .	No es para binarios: Pesa mucho y es ineficiente si intentas guardar videos gigantes o bases de datos.

Una vez que elegimos la plataforma, el trabajo colaborativo se vuelve central. Para que un compañero pueda subir código a tu repositorio, debes invitarlo siguiendo estos pasos en GitHub:

1. En tu repo de **GitHub**, ve a la pestaña **Settings** (Ajustes).
2. Menú lateral izquierdo: **Collaborators**.
3. Botón **Add people** y busca su nombre de usuario.
4. Una vez que acepte la invitación, ¡podrán programar juntos!

Flujo local y flujo remoto:

Ahora bien, para interactuar con estas plataformas en el día a día, existen dos caminos según cómo empiece el proyecto.

1. Flujo local: El primero es cuando la idea nace en tu PC, ya que no siempre empezamos descargando algo de internet. La ruta crítica de creación desde cero consiste en:

1. **Crear Carpeta:** Crea una carpeta para tu proyecto en tu PC.
2. **Entrar:** Abre esa carpeta con VS Code.
3. **Inicializar:** En la terminal escribe `git init`. (Esto crea el "ojo de **Git**" invisible en tu carpeta).
4. **Primer Paso:** Crea tu archivo `index.html`.
5. **Add & Commit:** Haz tu primer `git add .` y `git commit -m "Inicio de proyecto"`.
6. **Push a GitHub:** Haz tu primer push `git push -u origin master`.

2. Flujo remoto: Si tu profesor ya creó el repo, el proceso es aún más fácil:

1. **Cloing (Clonar):** En lugar de `git init`, usas el botón verde "Code" en **GitHub** y copias la URL.
2. **Terminal:** Escribes `git clone [URL]`. (Esto descarga la carpeta y todo su historial).
3. **Trabajar:** Abres la carpeta, editas tu `index.html`.
4. **Push:** Haces `git push origin master` para subir tus cambios.



TIP

¿Por qué -u? El comando `git push -u origin master` le dice a **Git**: "Subí esto por primera vez y recordá de ahora en adelante que la carpeta de internet se llama `origin` y la rama principal `master`". -U significa upstream, es decir corriente arriba.

Dominar estos flujos y entender el comportamiento de las plataformas colaborativas te dará la base operativa necesaria para integrarte a cualquier equipo de desarrollo profesional.

1.2. El ADN de un Cambio: El COMMIT y el SHA-1

Cada vez que estás satisfecho con un cambio en tu código, le tomas una "foto" usando un comando llamado `commit`. Cuando haces esto, **Git** hace algo increíble:

1. **Genera un Hash (SHA-1):** Es un código único e indescifrable de 40 caracteres (ej: `1a2b3c4...`). Es el "DNI" o huella digital de ese cambio. No hay dos iguales en el mundo.
2. **Snapshot (Instantánea):** A diferencia de otros programas que solo guardan la diferencia de lo que cambió, **Git** guarda una foto completa de cómo se veía el proyecto en ese segundo exacto.

En lenguaje simple: Un commit es como guardar una partida en un videojuego. Si algo sale mal más adelante, siempre puedes volver a este punto exacto donde todo funcionaba.

Traducción Técnica: **Git** utiliza el algoritmo **SHA-1** para garantizar la integridad de los datos. Cada commit es un objeto en la base de datos de **Git** que apunta a un 'tree' (directorio) y contiene metadatos como el autor, la fecha y el hash del commit padre. Esto crea una cadena de bloques inmutable (DAG - Directed Acyclic Graph).

| Comparativa: **Git** vs **GitHub**

Es el error número uno de los principiantes confundir estos dos términos. Son cosas completamente distintas:

Característica	<u>Git</u> (La Herramienta)	<u>GitHub</u> (El Servicio)
¿Qué es?	Un programa o software local de control de versiones.	Una plataforma web de alojamiento (hosting) en la nube.
Internet	No requiere internet. Funciona puramente en tu PC.	Sí requiere internet para subir, bajar o ver código.
Analogía	Es la cámara de fotos que usas en tus vacaciones.	Es Instagram , la red social donde subes tus fotos para que otros las vean.

Tema 2: Inicialización de repositorios, áreas de trabajo y commits.

2.1. Puesta en marcha del entorno local y el flujo de trabajo profesional

Con las bases teóricas de la arquitectura distribuida comprendidas, es el momento de abrir nuestra consola e iniciar la configuración operativa en nuestra computadora para interactuar con el sistema de versiones de forma estandarizada.

PASO 1

Antes de empezar a guardar nuestro código, debemos "presentarnos" ante **Git**. Si no lo hacemos, **Git** no sabrá de quién es el código que se está guardando y nos dará error.

Abre tu programa **Git Bash** (la terminal de consola negra que instalaste en la Clase 1) y escribe exactamente estos comandos uno por uno, presionando la tecla **Enter** después de cada línea:

```
# 1. Configurar tu identidad (Usa tus datos reales)
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"

# 2. Configurar la rama principal moderna
# Históricamente se llamaba 'master'. Hoy el estándar de la industria es
llamarla 'main'.
```

```
git config --global init.defaultBranch main
```

PASO 2

Una vez completada la identificación, debemos comprender cómo Git administra internamente la información. Para que **Git** guarde tus cambios, el código no se guarda automáticamente al presionar **Ctrl + S**. El código debe pasar intencionalmente por tres "zonas". Imagina que estás cocinando en un restaurante:

1. **Working Directory (La Heladera):** Aquí están tus archivos sueltos. Estás escribiendo código, probando cosas en VS Code. **Git** los ve pero no los ha asegurado todavía. (*Estado técnico: Untracked / Modified*).
2. **Staging Area (La Bandeja de Preparación):** Es una zona técnica intermedia. Sirve para revisar: "*¿Realmente quiero guardar todos estos archivos en el historial?*". Puedes elegir subir solo el archivo `index.html` a la bandeja y dejar tu archivo **CSS** en la heladera por ahora. (*Estado técnico: Staged*).
3. **Repository (El Plato Terminado):** Una vez que el plato sale de la bandeja, le sacas la foto definitiva (`commit`). El cambio está listo y "sellado" permanentemente en la base de datos interna de **Git** con su DNI único. (*Estado técnico: Committed*).

PASO 3

Para interactuar entre estas tres zonas, utilizaremos los comandos de terminal que usarás todos los días de tu vida profesional. Memorízalos:

Comando (Escribir en <u>Git</u> Bash)	¿Qué hace realmente?
<code>git init</code>	" ¡Git, empieza a vigilar esta carpeta! ". Convierte una carpeta normal en un Repositorio (crea el archivo oculto <code>.git</code>). Se ejecuta una sola vez por proyecto.
<code>git status</code>	" ¿Cómo vamos? ". Te dice qué archivos están "sucios" en rojo (en la heladera) o listos en verde (en la bandeja).
<code>git add .</code>	" Pon todo en la bandeja ". Prepara todos los archivos modificados a la vez (el punto <code>.</code> significa "todo en esta carpeta").
<code>git restore --staged <archivo></code>	" Oops, sácalo de la bandeja ". Si hiciste <code>add</code> por error a un archivo que no querías, esto lo devuelve a la heladera (rojo) sin borrar tu código.
<code>git commit -m "mensaje"</code>	" Saca la foto y guárdala ". Sella los archivos de la bandeja. El mensaje <code>-m</code> debe explicar claramente QUÉ cambiaste.
<code>git log --oneline</code>	" Muéstrame el álbum de fotos ". Lista todos los commits que has hecho hacia atrás, con sus respectivos Hashes (DNI).

2.2. Conectando con la Nube (GitHub)

Tu código está a salvo en tu computadora local gracias a **Git**, pero si tu disco duro se rompe, lo pierdes todo. Necesitamos un puente hacia la nube: **GitHub**.

Pasos para el primer Despliegue (Deploy) a GitHub: 1. Crea una cuenta gratuita en github.com.

1. Arriba a la derecha de la web, haz clic en el botón **+** y selecciona **"New repository"**. Ponle un nombre, déjalo público y NO marques ninguna otra casilla. Dale a "Create".
2. **GitHub** te dará unos comandos de ayuda. Ve a la terminal **Git** Bash de tu proyecto en tu PC y escribe:

```
# 1. Conectar tu carpeta local con el servidor en la nube de GitHub
git remote add origin https://github.com/tu-usuario/tu-repo.git

# 2. Asegurar que la rama principal local se llame igual que en la nube (main)
git branch -M main

# 3. Empujar (Push) todos tus commits históricos locales hacia la nube
git push -u origin main
```

Flujo Diario (El Pan Nuestro de Cada Día): Una vez que hiciste esa conexión inicial por primera vez, tu rutina de trabajo cada tarde será repetitiva y sencilla:

1. Haces cambios en VS Code y los preparas: `git add .`
2. Sellas la foto localmente: `git commit -m "feat: agregué el título principal"`
3. Lo subes a la nube para asegurarlo: `git push`

2.3. Buenas Prácticas para Commits

Los mensajes de tus commits son para tus compañeros de equipo (¡y para tu "yo" del futuro que intentará entender qué diablos hizo hace un mes!). En la industria usamos "Prefijos Convencionales".

- **✗ Pésimo:** `git commit -m "cosas"`, `git commit -m "arreglé un bug"`

- **Profesional:**

- **feat:** para nuevas funcionalidades (Ej: **feat:** agregué el carrito de compras).
- **fix:** para corrección de errores (Ej: **fix:** reparé el color del botón que no se veía).
- **docs:** para cambios en textos o documentación (Ej: **docs:** actualicé el archivo README).

Documentación Oficial (Referencias)

- **CSS:** <https://developer.mozilla.org/es/docs/Web/CSS>
- **Git:** <https://git-scm.com/doc>
- **GitHub:** <https://docs.github.com/es>
- **HTML:** <https://developer.mozilla.org/es/docs/Web/HTML>